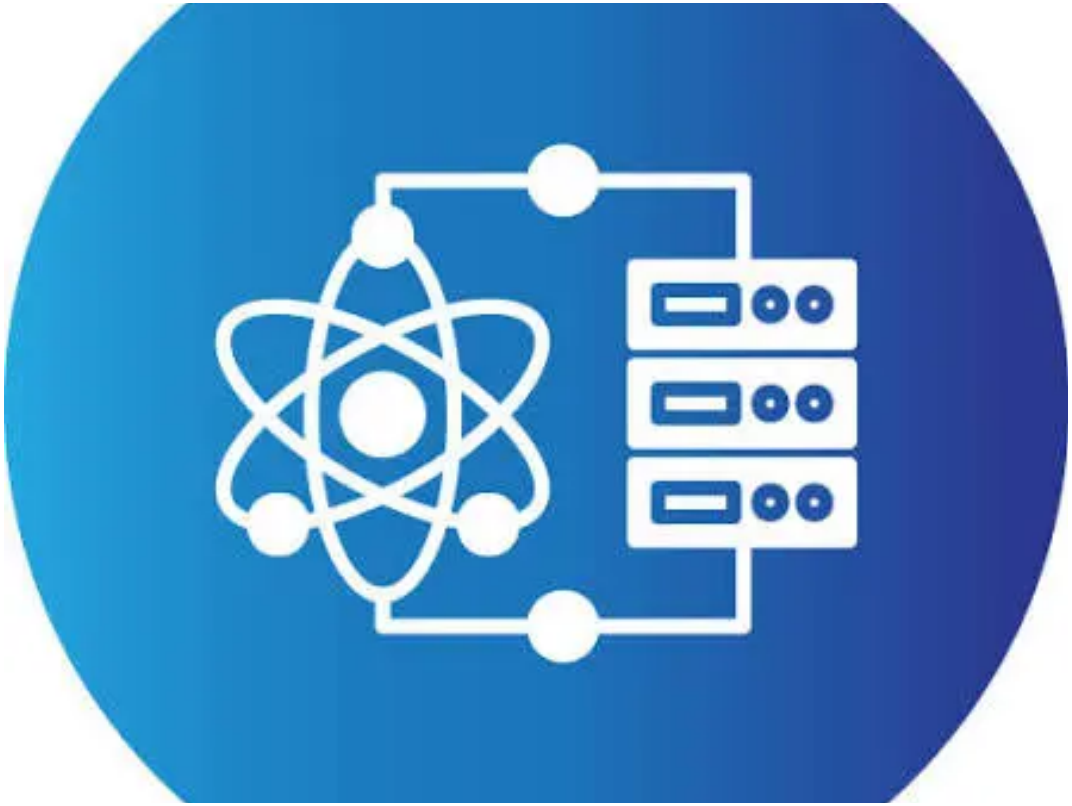


DATA SCIENCE V1.0

Lesson 1

Documentation: Data Science V1.0

Python for Data Science & API Foundations

Objectives

- Build strong Python foundation
- Understand how ML models are exposed in production

- Learn REST APIs and containerization basics
- Introduce software engineering discipline (structure, logging, validation)

Topics

- Python syntax, variables, control flow
- Data structures (list, dict, tuple, set)
- Functions & OOP
- Virtual environments (venv, pip)
- NumPy fundamentals
- REST APIs
- FastAPI vs Flask
- WSGI vs ASGI
- Docker fundamentals
- Logging best practices

Resources

- **Python**
  - https://docs.python.org/3/tutorial/
  - https://realpython.com/
- **NumPy**
  - https://numpy.org/doc/stable/
- **FastAPI**
  - https://fastapi.tiangolo.com/
- **Flask**
  - https://flask.palletsprojects.com/
- **Docker**
  - https://docs.docker.com/get-started/
- **Practice Platform**
  - https://www.kaggle.com/learn/python

Practice Project – Production-Style ML Scoring API

Business Scenario

- You work for a fintech startup.
- You must expose a scoring API that evaluates loan applications based on simple risk rules.

Functional Requirements: Build a FastAPI application with:

- Endpoint:
- POST /predict
- Input JSON:

```
{
  "age": 35,
  "income": 50000,
  "loan_amount": 10000,
  "credit_score": 680
}
```

- Output JSON:

```
{
  "risk_score": 0.42,
  "risk_category": "LOW"
}
```

- Risk logic:
  - Normalize features manually
  - Create weighted scoring formula
  - Categorize:
    - 0–0.3 → LOW
    - 0.3–0.7 → MEDIUM
    - 0.7 → HIGH

Technical Requirements

- Use Pydantic models

- Input validation
- Logging to file + console
- Error handling (invalid input)
- Environment variables for config
- Dockerized application
- Requirements.txt
- README.md

Suggested Structure

```
app/
├── main.py
├── models.py
├── services/
│   └── scoring_service.py
├── utils/
│   └── logger.py
└── config.py
Dockerfile
requirements.txt
README.md
```